In Class Lab 5h

Variables and Decisions | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 90 minutes	lab5h_starter.sql	Your GitHub repository URL

Textbook: Coronel and Morris, *Database Systems: Design, Implementation, and Management*, 14th edition, Chapter 5, Section 5.8.
The due date is listed on Canvas LMS.

What this lab adds to Lab 5g

Lab 5g ended with two problems. add_score happily stored a score of minus 20, and record_attendance answered a missing student with a foreign key error meant for a database administrator. Both procedures did exactly what they were told, because they had no way to **look before they acted**.

Looking before acting needs two new tools. A **variable** holds something the procedure found out, such as how many students have a given id. A **decision**, written with IF, chooses what to do next based on it. By the end of this lab both Lab 5g procedures check their input and say no politely.

New in this lab	Steps
Session variables, the ones that start with @	3 and 4
Local variables, made with DECLARE inside a procedure	5
OUT parameters, which hand a value back to whoever called	6
Why the data type of a variable matters, shown by a real bug	7
IF, ELSEIF and ELSE	8 to 10

Two kinds of variable

This is the idea the whole lab rests on, so it is worth reading twice.

	Session variable	Local variable
Looks like	@passing	v_total
Created by	Just using it. SET @passing = 80 makes it	DECLARE, at the top of a BEGIN block
Lives until	Your connection closes	The procedure reaches END
Can be seen from	Anywhere in your connection, including inside a procedure	Only inside the procedure that declared it
If you never set it	It is NULL, with no error	It is NULL, or its DEFAULT value

Part 1. Get set up

Step 1. Connect to information_schema and make your folder

Make a folder on OneDrive called **CIS4720_Lab5h**, download **lab5h_starter.sql** from Canvas into it, and open the folder in VS Code. Connect SQLTools to **information_schema** as root, exactly as in Labs 5f and 5g.

Step 2. Create the schema and run the starter file

Run **show databases;**, **create database if not exists cis_students_db;** and **use cis_students_db;** one at a time. Then run the rest of the file one statement at a time, selecting each function, trigger and procedure as a whole block.

> **One connection is enough**
>
> The **use cis_students_db;** line switches the connection you already have over to your new schema, so you can stay in that connection for the whole lab. You never need the command prompt to create a schema: connect to information_schema, which is always there, and create yours from inside VS Code.
>
> A second SQLTools connection with Database set to **cis_students_db** is optional. Make one if your statements ever land in the wrong place, which you can check with **SELECT DATABASE();**. It only has to say the name of your schema.

Check	What you should see
CALL list_students();	92.33, 88.00, 85.00, 84.00, 82.50, highest first
SELECT * FROM attendance;	Five rows, ids 500 to 504

Part 2. Session variables

Step 3. Make some and use them

These are printed for you because every piece is new. Copy them under the Step 3 heading and run them in order.

```
-- make a session variable and look at it
SET @passing = 80;
SELECT @passing;

-- use it in a query, anywhere a value could go
SELECT student_name, test_name, score
FROM student_scores
WHERE score < @passing;

-- a variable nobody ever set
SELECT @never_set;

-- put the answer to a query straight into a variable
SELECT COUNT(*) INTO @how_many FROM students;
SELECT @how_many;
```

Statement	What comes back
SELECT @passing	80
The query using @passing	Two rows: Robert Reese, quiz2, 72 and Inder Singh, quiz3, 74
SELECT @never_set	NULL. No error at all
SELECT @how_many	5

Two things that catch people

A misspelled session variable is not an error. If you SET @passing and then SELECT @pasing, you get NULL, the same as @never_set. When a variable is unexpectedly NULL, check the spelling first.

It lives in your connection. When a step has a CALL followed by a SELECT of a variable, select both lines and run them together as one selection. That way they are sure to use the same connection, whichever way VS Code is set up.

Step 4. Session variables on your own

You write all of these. Steps 3 and Lab 5f are your models.

1. Use SELECT ... INTO to put the **highest score on quiz3** into a variable called **@top**, then show it. It should be **95**.
2. SET a variable called **@test** to the text quiz3, then call scores_for_test, passing **@test** instead of typing the test name.
3. SET **@who** to 3 and call student_report with it.

Call	What comes back
scores_for_test(@test)	Robert Reese 95, Maria Elena Jimenez 94, Inder Singh 74
student_report(@who)	Maria Elena Jimenez: quiz1 95, quiz2 88, quiz3 94

A procedure does not care whether its value was typed in or came from a variable. That is what makes the next idea possible: a procedure that hands a value back.

Part 3. Local variables and OUT parameters

Step 5. Local variables, class_stats

Session variables belong to you. **Local** variables belong to one procedure. They are made with **DECLARE**, used while the procedure runs, and thrown away at END. This one is printed in full.

```
-- three numbers about the whole class, worked out one at a time
CREATE PROCEDURE class_stats()
BEGIN
    -- DECLARE makes a local variable: a name, a type, and a starting value
    DECLARE v_students INT DEFAULT 0;
    DECLARE v_scores INT DEFAULT 0;
    DECLARE v_average DECIMAL(5,2);

    -- fill the first one
    SELECT COUNT(*) INTO v_students FROM students;

    -- one SELECT can fill two variables, in the same order
    SELECT COUNT(*), ROUND(AVG(score), 2)
    INTO v_scores, v_average
    FROM test_scores;

    -- show all three as one row
    SELECT v_students AS students, v_scores AS scores, v_average AS class_average;
END
```

Call it. One row comes back: **5** students, **11** scores, class average **86.55**.

Now prove the local variables are gone. Under the CALL, run **SELECT v_students;** on its own. The server answers **ERROR 1054, unknown column 'v_students'**. Outside the procedure the name means nothing, so the server guesses it must be a column. Put that error in a comment.

> **The rules for DECLARE**
>
> Every DECLARE goes at the top of the BEGIN block, before any other statement. Put one lower down and you get error 1064.
>
> A local variable needs a type, just like a column. **DEFAULT** is optional. Without it the variable starts as NULL.
>
> The v_ at the front is a habit, like p_ for parameters. It stops a variable being mistaken for a column with the same name, which is a real and confusing bug.

Step 6. An OUT parameter, count_scores

Every parameter so far has been IN: a value travelling into the procedure. An **OUT** parameter travels the other way. The procedure puts a value in it, and when the CALL finishes the value lands in the session variable you named.

```
-- counts one student's scores and hands the number back
CREATE PROCEDURE count_scores(IN p_student_id INT, OUT p_count INT)
BEGIN
    -- count this student's rows and put the answer in p_count
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<
END
```

Writing the statement that goes in the gap

One SELECT ... INTO, the same shape as the first statement in class_stats.

1. SELECT **COUNT(*)**.

2. INTO p_count. An OUT parameter is filled exactly like a local variable.

3. FROM test_scores, WHERE student_id equals p_student_id. Then a semicolon.

To call it, give a **session variable** in the OUT position, then look at it. Run each pair together as one selection:

```
CALL count_scores(3, @n);
SELECT @n;
```

CALL for	@n afterwards
Student 3	3
Student 5	1, Rita's quiz4
Student 9	0, because nobody has that id

Try **CALL count_scores(3, 5);** once and note the error. The server needs somewhere to put the answer, and a plain 5 is not somewhere: **ERROR 1414, OUT or INOUT argument 2 is not a variable**.

There is a third kind: INOUT

An INOUT parameter carries a value in **and** back out changed. For example a procedure declared with **INOUT p_score DECIMAL(5,2)** whose body is **SET p_score = p_score + 5;** turns @sc from 80 into 85 each time you CALL it with @sc. You will not need INOUT in these labs, but you will see it in examples.

Step 7. When a local variable is too small

This step is about a bug that has been sitting in your database since Lab 5d. Nobody has hit it yet only because nobody has scored a perfect average. Most of the writing in this step is yours.

a. Add a new student, **Ana Lopez**, with an INSERT that names only first_name and last_name. She becomes student **7**, not 6, because James Toor used 6 back in Lab 5d and the counter never goes back.

b. Give Ana a quiz1 score of **100** using add_score. It fails: **ERROR 1264, out of range value for column 'v_gpa'**.

c. Now look at Ana's rows in test_scores, and at her gpa in students. The score of 100 **is there**. Her gpa is still empty.

What just happened

Inside calculate_gpa, the answer goes into **DECLARE v_gpa DECIMAL(4,2)**. DECIMAL(4,2) means four digits in all, two of them after the point, so the biggest it can hold is **99.99**. An average of 100.00 needs five digits.

add_score ran its INSERT, which worked, then its UPDATE, which called the function, which failed. A procedure is **not all or nothing**. Every statement that finished before the error stays done. Making a group of statements succeed or fail together is a later topic called transactions.

d. Fix the function. Drop calculate_gpa, then create it again exactly as in your starter file with **two** changes: DECIMAL(4,2) becomes DECIMAL(5,2) in the RETURNS line and in the DECLARE line.

e. Do not call add_score again, because the score is already stored and you would store it twice. Instead write an UPDATE that sets student 7's gpa to calculate_gpa(7). It fails **again**, with a different message: **out of range value for column 'gpa'**.

f. The column has the same problem as the variable did. students.gpa was created as FLOAT(4,2) in Lab 5d. Use **ALTER TABLE ... MODIFY** to change it to **DECIMAL(5,2)**, then run your UPDATE again. Call list_students: Ana is at the top with **100.00**.

The lesson is worth more than the fix. A variable's type is a promise about what it will hold, and the server keeps that promise by refusing anything bigger. When you DECLARE a variable to hold a column's value, give it the same type as the column, or a bigger one.

Part 4. Decisions with IF

Step 8. IF, ELSEIF and ELSE, student_standing

This procedure looks up a student's gpa into a local variable, decides which of four labels fits, and hands the label back through an OUT parameter. Everything but the decision is written for you.

```
-- reports a student's standing as a word, through an OUT parameter
CREATE PROCEDURE student_standing(IN p_student_id INT,
                                  OUT p_standing VARCHAR(20))
BEGIN
    -- the gpa goes here once we have looked it up
    DECLARE v_gpa DECIMAL(5,2);

    -- look it up. no matching student leaves v_gpa NULL
    SELECT gpa
    INTO v_gpa
    FROM students
    WHERE student_id = p_student_id;

    -- choose the label
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<
END
```

Writing the IF block that goes in the gap

Four branches, checked from top to bottom. Each one sets p_standing.

1. IF v_gpa IS NULL THEN, and on the next line **SET p_standing = 'No such student';**. This catches an id that matched nothing.

2. ELSEIF v_gpa >= 90 THEN, then set it to 'Honors'.

3. ELSEIF v_gpa >= 85 THEN, then 'Good standing'. One word, ELSEIF, with no space. ELSE IF with a space starts a second IF that needs its own END IF.

4. ELSE with no condition, then 'Needs support'. ELSE catches everything left.

5. END IF; to close the whole thing. One END IF, however many ELSEIFs.

Call it for students 7, 3, 1, 4 and 9, each with a SELECT of the OUT variable, run together. You write these.

Student	gpa	@standing
7, Ana Lopez	100.00	Honors
3, Maria Elena Jimenez	92.33	Honors
1, Jenna Wilson	85.00	Good standing
4, Inder Singh	82.50	Needs support
9	none	No such student

Order matters

The server takes the **first** branch that is true and skips the rest. Jenna's 85.00 is not 90 or more, so the second branch is skipped, and it is 85 or more, so she gets Good standing.

Swap the two ELSEIF lines and Maria Elena's 92.33 would stop at the 85 test and come out as Good standing. Check the most demanding condition first.

CASE does the same job in a different shape: **CASE WHEN v_gpa >= 90 THEN ... WHEN v_gpa >= 85 THEN ... ELSE ... END CASE;**. Either is fine. These labs use IF.

Step 9. Rebuild add_score so it checks the score

Lab 5g's first problem. Write the DROP for add_score yourself, then copy this skeleton and fill its two gaps.

```
-- adds a score only when it is a real score
CREATE PROCEDURE add_score(IN p_test_name VARCHAR(20),
                           IN p_student_id INT,
                           IN p_score FLOAT)
BEGIN
    IF  -- >>>>>  GAP 1: THE CONDITION FOR A BAD SCORE  <<<<<
    THEN
        -- refuse, and say why
        SELECT 'Score must be between 0 and 100' AS message;
    ELSE
        -- >>>>>  GAP 2: THE INSERT AND THE UPDATE FROM LAB 5G  <<<<<
    END IF;
END
```

Filling the two gaps

Gap 1. A score is bad when it is below 0 **or** above 100. Write those two comparisons on p_score joined by **OR**. Not AND: no score is below 0 and above 100 at once, so an AND condition would never be true and nothing would ever be refused.

Gap 2. Your INSERT and your UPDATE from Lab 5g, unchanged, in the same order. Everything between ELSE and END IF runs only when the score passed the check.

Test it with three calls you write: quiz5 for student 4 at **minus 20**, then at **120**, then quiz5 for student 2 at **91**.

Call	What happens
Student 4, minus 20	A one row result: Score must be between 0 and 100. Inder stays at 82.50
Student 4, 120	The same message. Still no quiz5 rows at all
Student 2, 91	Accepted. Robert goes from 84.00 to 85.75

Step 10. Rebuild record_attendance so it checks the student

Lab 5g's second problem, and yours from start to finish. The new idea is that the procedure looks something up first, keeps the answer in a local variable, and decides based on it.

Walking through it

1. Drop record_attendance. Start the new one with the same name and the same two IN parameters as Lab 5g.

2. After BEGIN, DECLARE a local variable **v_found**, INT, DEFAULT 0.

3. A SELECT COUNT(*) INTO v_found FROM students WHERE student_id equals the parameter. It will be 1 if the student exists and 0 if not.

4. IF v_found = 0 THEN SELECT the message **'No such student'** AS message.

5. ELSE your Lab 5g INSERT, unchanged. Then END IF; and END.

Why COUNT and not the gpa trick from Step 8? COUNT always returns exactly one row, 0 or more, so v_found is never NULL. That makes the IF simple.

Test it: record student **9** as present, then student **3** as **late**, then select the attendance table.

Call	What happens
Student 9	A one row result: No such student. No error 1452 this time
Student 3, late	Row 505, today's date, LATE in capitals from the Lab 5e trigger

Both Lab 5g problems are fixed. One thing is still not quite right, and Project 11 deals with it: a message in a result looks like success to a program calling the procedure. There is a way to make a procedure refuse with a real error of its own.

Part 5. Push to GitHub

Step 11. Publish to a new repository

Save lab5h_starter.sql first. If you are not signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize**.

1. Click **Source Control**, then **Publish to GitHub**.
2. Name it **cis4720-lab5h**. It has to be a name you have not used.
3. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
4. Tick **lab5h_starter.sql** and nothing else. Leave the **.vscode** folder unticked, because that is the one holding your password. If you see a file ending in **.session.sql**, leave it unticked too. It is SQLTools' scratch pad, not your work, and only lab5h_starter.sql is graded.

Sign out of GitHub before you leave the lab

Shared PC. Your sign in stays on it until somebody removes it. Click **Accounts**, then your username, then **Sign Out**. Everything else on the PC is wiped tonight, so the repository is the only copy of your work.

Step 12. Check it and submit the URL

1. Open the repository on github.com, click the file and read it on the page. What is there is exactly what gets graded.
2. Open the URL in a private or incognito window. A **404** means it is private, so go to **Settings** and change the visibility to public.
3. Paste the URL into the Lab 5h submission box on Canvas.

Submit the URL even if you did not finish. Partial work your instructor can read scores better than finished work nobody can reach.

If you are reading procedural SQL somewhere else

Every database writes procedural SQL a little differently. If an example in the textbook or online uses Oracle's PL/SQL, these are the MySQL equivalents of what you used today:

Oracle PL/SQL	MySQL, as in these labs
Variables declared in a DECLARE section before BEGIN	DECLARE inside BEGIN, at the top
v_total := 0;	SET v_total = 0;
DBMS_OUTPUT.PUT_LINE('text');	SELECT 'text' AS message;
IF ... THEN ... ELSIF ... ELSE ... END IF;	IF ... THEN ... ELSEIF ... ELSE ... END IF;
EXEC procedure_name(...);	CALL procedure_name(...);

How it is graded

Steps	What is being checked	Points
1 to 2	Schema created and the starter file running	1.0
3	The session variable statements run	0.5
4	@top, @test and @who written and used	1.0
5	class_stats created and called, with the 1054 error noted	0.5
6	count_scores created, with the SELECT INTO you wrote, called for 3, 5 and 9	1.0
7	Ana added, both 1264 errors noted, the function and the column fixed, Ana at 100.00	1.5
8	student_standing created, with the IF block you wrote, called five times	1.5
9	add_score rebuilt with the check and tested three ways	1.0
10	record_attendance rebuilt from the walkthrough and tested twice	1.0
All	Required. A comment in your own words above every statement	1.0
11 to 12	A new public repository with the file in it, URL submitted	0.0

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

What you see	What it means
ERROR 1064 near 'DELIMITER $$'	Delete the DELIMITER lines and the $$ after END.
ERROR 1064 near '--something'	A comment lost the space after its two dashes.
ERROR 1064 on a DECLARE line	A DECLARE comes after another statement. Move every DECLARE to the top of the BEGIN block.
ERROR 1064 near END at the bottom	ELSE IF written as two words. Make it ELSEIF, one word.
ERROR 1054, unknown column 'v_...'	A local variable used outside its procedure, or spelled differently from its DECLARE.
SELECT @n shows NULL	The CALL and the SELECT ran on different connections, or the variable is misspelled. Run them together as one selection and check the spelling.
ERROR 1414, argument is not a variable	An OUT parameter was given a plain value. Give it a session variable such as @n.
ERROR 1264, out of range value	A value is too big for its variable or column. Expected in Step 7, nowhere else.
ERROR 1304, already exists	Drop the procedure or function first, then create it again.
Every score is refused in Step 9	The comparisons in Gap 1 point the wrong way. Bad means less than 0 or greater than 100.
Nothing is ever refused in Step 9	Gap 1 uses AND. No score is below 0 and above 100 at once.